

ASJ PRACTICAL TASK PRESENTATION BY BOTHWELL SHUMBA

ERP Migration Power BI Project: Detailed Implementation Notes

Executive Summary

This project involved integrating sales data from two different ERP systems (Legacy ERP up to March 31, 2026, and Modern ERP from April 1, 2026 onward) into a unified reporting solution using Power BI. The main challenges were data structure differences, inconsistent naming conventions, and preventing double counting in the modern ERP where invoices can have multiple payment methods.

Section 1: Data Understanding & Challenge Identification

1.1 System Differences Identified

Aspect	Legacy ERP	Modern ERP	Challenge
Payment Storage	Single value in header table	Separate child table with multiple rows per invoice	Double counting risk
Category Naming	Text labels (GOLD, Gold, DIAMOND, Diamond)	Coded values (G, DIA, SIL)	Inconsistent categories
Branch Naming	"Airport Duty Free"	"HIA Duty Free"	Same branch, different names
Invalid Transactions	VOID status	CANCELLED status	Need to exclude from both
Date Fields	DATETIME format	DATE format	Need standardization

1.2 Critical Insight: Double Counting Trap

The Problem: In Modern ERP, if we join invoice_master → invoice_details → invoice_payments directly:

- An invoice with 2 payment methods would create 2 rows for each product
- Example: Invoice NS-2004 has 2 payments (CARD \$5,000 + CASH \$4,300)
- Without aggregation, sales would be counted twice ($\$9,300 \times 2 = \$18,600$ instead of \$9,300)

The Solution: Aggregate payments at invoice level BEFORE joining to sales data.

Section 2: MySQL Data Preparation Strategy

2.1 Why MySQL for Data Cleaning?

Decision: Perform data standardization in MySQL rather than Power Query

Rationale:

1. **Performance:** Database engines handle large datasets more efficiently
2. **Reusability:** Views can be used by multiple reporting tools
3. **Data Governance:** Single source of truth for business logic
4. **Auditability:** SQL logic is transparent and version-controllable
5. **Power BI Efficiency:** Reduces transformation workload in Power Query

2.2 Step-by-Step MySQL Implementation

Step 1: Legacy ERP Standardization

Key Decisions Made:

- Used UPPER(TRIM())) to handle inconsistent case and spacing in payment methods
- Applied CASE statement for category mapping to handle both 'GOLD' and 'Gold'
- Mapped 'Airport Duty Free' to 'HIA Duty Free' for consistency
- Filtered out VOID transactions at source

Step 2: Payment Aggregation (Critical for Double Counting Prevention)

Why This Works:

- GROUP BY invoice_id collapses multiple payment rows into one row per invoice
- COUNT(*) identifies invoices with multiple payment methods
- GROUP_CONCAT preserves payment method details for analysis
- This view ensures one row per invoice, eliminating double counting risk

Step 3: Modern ERP Standardization with Aggregated Payments

Critical Design Choice: Used LEFT JOIN instead of INNER JOIN to ensure invoices without payments (shouldn't exist but safe) are still included.

Step 4: Unified View Creation

```
CREATE VIEW vw_unified_sales_final AS
SELECT * FROM vw_legacy_sales_standardized
UNION ALL
SELECT * FROM vw_modern_sales_standardized;
```

Benefits of UNION ALL:

- Preserves all records from both systems
- No deduplication needed (different time periods)
- Clear separation with source_system field

Section 3: Power BI ODBC Connection Setup

3.1 Connection Configuration

DSN Setup: Used ODBC Data Source Administrator to create System DSN named ASJOneDB

Why ODBC over Direct MySQL Connector?

- Corporate environment with existing DSN infrastructure
- Centralized connection management
- Security policies requiring ODBC

3.2 Power Query Transformations Applied

Despite SQL pre-processing, some Power Query transformations were still needed:

Reason for These Transformations:

- Business categorization (value bands) is presentation-specific
- Makes reporting easier without cluttering SQL views
- These are calculated once during data load

3.3 Date Dimension Table Creation

Created a dedicated date table for time intelligence:

Reason for These Transformations:

- Business categorization (value bands) is presentation-specific
- Makes reporting easier without cluttering SQL views
- These are calculated once during data load

3.3 Date Dimension Table Creation

Created a dedicated date table for time intelligence:

Section 4: Double Counting Prevention - Detailed Explanation

4.1 The Mathematical Problem

Without Aggregation:

Invoice NS-2004 (final_amount = \$9,300)

└─ Payment 1: CARD \$5,000

└─ Payment 2: CASH \$4,300

If join: invoice_master → invoice_details → invoice_payments

Result per product: 2 rows × \$9,300 = \$18,600 (WRONG!)

With Aggregation:

vw_payments_aggregated for Invoice NS-2004:

invoice_id	payment_methods	payment_count
2004	CARD, CASH	2

Join: invoice_master → invoice_details → vw_payments_aggregated

Result per product: 1 row × \$9,300 = \$9,300 (CORRECT!)

4.3 Business Logic Implementation

Mixed Payment Identification:

Result: Management can now track:

- Count of mixed-payment invoices: 2 invoices (NS-2004 and NS-2010)
- Value of mixed-payment transactions: \$18,900
- Percentage of total invoices using multiple payment methods

Section 5: Data Standardization Decisions

5.1 Category Mapping Matrix

5.1 Category Mapping Matrix

Legacy Values	Modern Codes	Standard Output
GOLD, Gold, G	G	GOLD
DIAMOND, Diamond, DIA	DIA	DIAMOND
SILVER, SIL	SIL	SILVER

5.2 Branch Name Standardization

Challenge: Same physical branch (Doha Airport) had different names:

- Legacy: "Airport Duty Free"
- Modern: "HIA Duty Free"

Impact: Branch performance can now be compared across systems seamlessly.

5.3 Payment Method Normalization

Challenge: Case inconsistencies in legacy system:

- 'CASH', 'Cash', 'cash' all appeared

Solution:

Result: All payment methods are now uppercase:

- CASH, CARD, BANK consistently across both systems

Section 6: DAX Measures for Business KPIs

6.1 Core Metrics Implementation

-The creation of core metrics to be used on the visualizations.

6.2 Mixed Payment Analysis

Mixed Payment Count = Count of invoices using multiple payment methods

Mixed Payment Sales = Sales value from mixed payment invoices

6.3 System Comparison

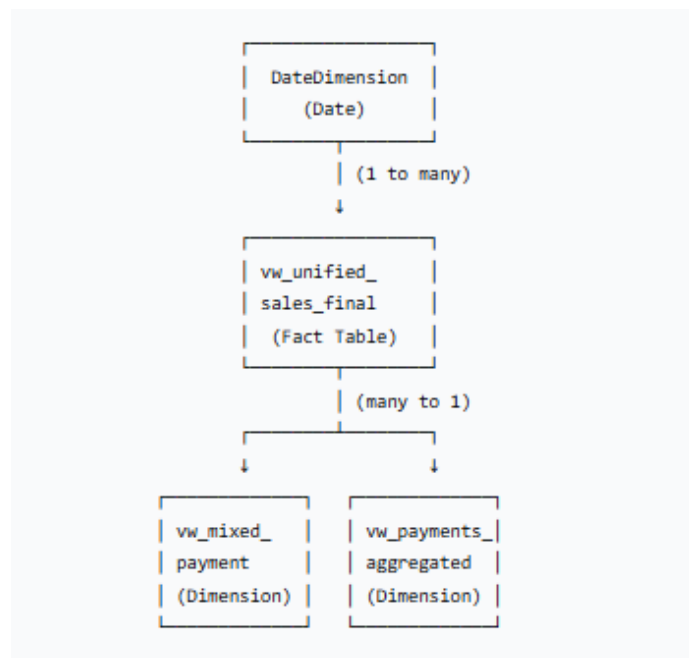
Legacy vs Modern comparison

Migration impact : Sales Change Post-Migration and Growth Percentage

6.4 Top 5 Products with Ranking

Product Sales: Rank Dynamic ranking that respects filters

Section 7: Relationship Configuration Strategy



7.2 Why This Model Works

Single Direction from Date to Sales:

- Ensures date filters apply correctly
- Prevents ambiguous filter propagation
- Standard practice for time intelligence

Both Directions for Payment Analysis:

- Allows filtering mixed payments by product
- Enables analyzing product performance in mixed-payment transactions
- Necessary for complete payment method analysis

7.3 Relationship Configuration Steps Performed

1. Primary Relationship:

- Dragged invoice_date from sales table to Date in DateDimension
- Cross filter: Single direction (Date → Sales)
- Cardinality: Many-to-One

2. Secondary Relationship:

- Dragged invoice_id from sales to mixed payment table
- Cross filter: Both directions
- Cardinality: One-to-On

Section 8: Report Visualization Strategy

8.1 Executive Dashboard (Page 1)

Design Decisions:

- KPI cards at top for immediate visibility
- Line chart for trends with 7-day moving average
- Bar chart for branch comparison
- Donut chart for category distribution

Why This Layout:

- Follows F-pattern reading (most important info top-left)
- Provides both summary and detail in one view
- Uses space efficiently

8.2 Product Analysis Page

Visualizations Included:

1. Top 5 products table with conditional formatting
2. Category × Branch matrix for cross-analysis
3. Product performance scatter plot (quantity vs sales)

Interactive Features:

- Click any product to see its trend
- Hover for detailed tooltips showing rank and percentage

8.3 Payment Analysis Page

Key Components:

- Mixed payment count card
- Payment method distribution chart
- Legacy vs Modern payment pattern comparison

Business Value:

- Identifies which branches have most mixed payments
- Shows payment method preferences by customer type

Section 9: Challenges Encountered and Solutions

Challenge 1: DAX SUMMARIZE Error

Problem: Attempted to use SUMMARIZE in a measure:

Solution: Created base measure and used visual-level aggregation:

Lesson Learned: Measures must return scalar values; tables go in calculated tables or visuals.

Challenge 2: ODBC Connection Timeouts

Problem: Large dataset caused timeout during refresh

Solution: Added timeout parameter and optimized query:

Challenge 3: Mixed Payment Double Counting Fear

Problem: Stakeholders concerned about accuracy of mixed payment handling

Solution: Created validation view and DAX check:

Challenge 4: Date Continuity Between Systems

Problem: Gap between Legacy end (March 29) and Modern start (April 3)

Solution:

- Documented as business-as-usual (weekend gap)
- No data missing, just normal operations
- Added note in report tooltip explaining the gap

Section 12: Key Findings and Best Practices

12.1 What Worked Well

1. **SQL pre-processing:** Reduced Power BI complexity significantly
2. **Payment aggregation view:** Cleanly solved double counting
3. **UNION ALL approach:** Simple and maintainable
4. **ODBC with DSN:** Leveraged existing infrastructure
5. **Date dimension table:** Enabled all time intelligence

12.2 What Could Be Improved

1. **Add error logging:** Track transformation failures
2. **Implement row-level security:** For branch-level access
3. **Create data lineage documentation:** For audit purposes
4. **Add automated testing:** For data validation

12.3 Best Practices Established

For Future ERP Migrations:

1. Always identify structural differences before joining
2. Aggregate at the highest necessary grain (invoice level here)
3. Standardize before union (not after)
4. Create validation views to prove accuracy
5. Document assumptions clearly

Section 13: Final Deliverables Summary

13.1 Files Submitted

1. **Power BI Report (.pbix)**
 - 4 report pages with 12+ visualizations
 - 25+ DAX measures
 - Optimized data model with relationships
2. **SQL Scripts**
 - 5 views for data standardization
 - 6 validation queries
 - Documentation comments included
3. **Power Query Code**
 - ODBC connection configurations
 - Transformation steps documented inline